

FACE RECOGNITION SYSTEM

SYSTEM DESIGN SPECIFICATION

Robust, Real-Time, Privacy-Aware, Multi-Camera and Edge-Ready Architecture

Version 1.0 | September 2026 | Design Baseline

Purpose: implementation blueprint before production coding begins.

1. Executive Summary

This specification defines a production-oriented face recognition platform whose primary engineering objective is reliable identity recognition under real-world variation: low light, pose, occlusion, blur, camera changes, beard and hairstyle changes, aging, weight change, and different image sources. The design deliberately treats face recognition as a multi-stage biometric pipeline rather than a single model.

The proposed system separates computer vision inference from application/business logic. Python owns model inference and vision processing; Java/Spring Boot owns APIs, identity management, security, persistence, camera/device management, and audit workflows. The design is intended to support a desktop prototype first, then real-time video, edge inference, and multi-camera scale without changing the core recognition contract.

A central principle is that “accuracy” is not a single percentage. The system will be evaluated using verification and identification metrics, operating-point metrics such as TAR at a specified FAR, latency/FPS, reject/unknown behavior, and a controlled robustness matrix. NIST evaluation practice similarly distinguishes false-positive and false-negative behavior and evaluates quality, pose, elapsed time, and other operational conditions. [R4-R6]

2. Problem Statement

The target problem is to identify or verify people from still images and real-time video while reducing false matches and maintaining identity stability when appearance, image quality, viewpoint, and capture device change. The design must also support unknown-person rejection, liveness/anti-spoofing as a security layer, privacy controls, model/version traceability, and future multi-camera deployment.

3. Goals and Success Criteria

Goal	Success definition
Recognition robustness	Same-person matching remains reliable across controlled appearance and capture variations.
Low false acceptance	Unknown people are rejected rather than assigned to the nearest enrolled identity.
Real-time behavior	Pipeline meets an agreed per-camera latency/FPS target on the chosen hardware.
Stable video identity	Temporal evidence prevents frame-to-frame identity flicker.
Scalability	Gallery search and inference architecture can scale beyond a small prototype.
Privacy	Raw images are minimized; biometric records are access-controlled, auditable, and deletable.
Reproducibility	Every benchmark result records dataset split, model version, thresholds, hardware, and configuration.
Deployability	Inference can be exported to ONNX Runtime and later optimized for edge hardware where supported.

4. Scope

4.1 In Scope

- Face detection, landmark localization/alignment, face embedding, similarity matching, unknown rejection, and temporal decisioning.
- Multiple enrollment samples per person and identity-template management.

- Image-quality assessment using blur, exposure, pose, face size, occlusion and related signals.
- Real-time video processing and multi-face tracking.
- Optional liveness/anti-spoofing module as a security layer.
- Python inference service, Java/Spring Boot application layer, PostgreSQL persistence and vector search.
- Benchmarking, threshold calibration, model selection, regression testing and operational telemetry.

4.2 Out of Scope for the First Release

- Custom training of a foundation-scale face recognition model from scratch.
- Automated demographic inference such as race, ethnicity, religion, or similar sensitive attributes.
- Uncontrolled public-space surveillance use cases.
- Legal/compliance certification for any jurisdiction. Compliance must be assessed for the actual deployment context.
- Fully distributed Kubernetes-scale deployment before the single-camera system meets the accuracy and latency gates.

5. Functional Requirements

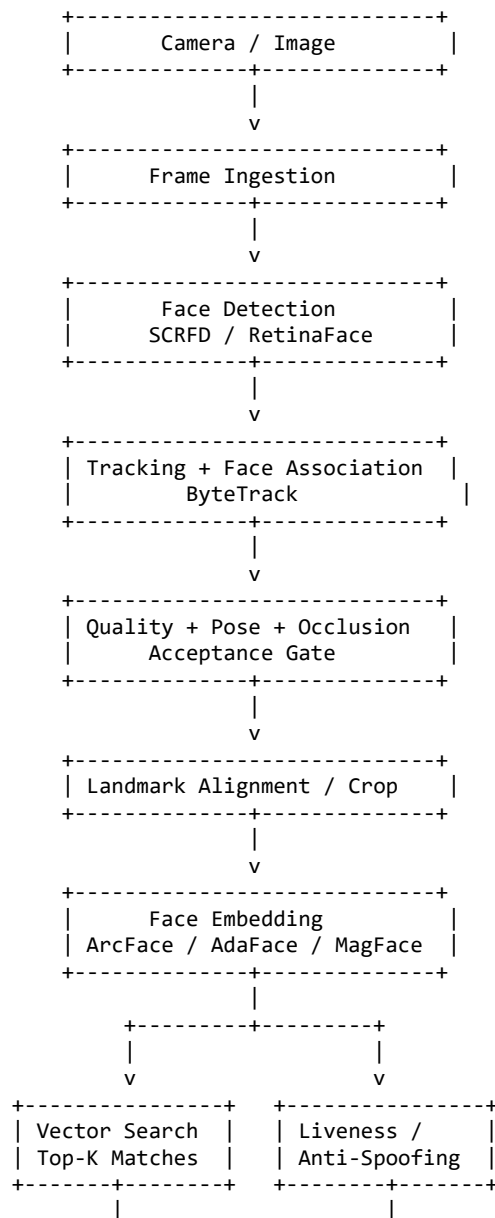
ID	Area	Requirement
FR-01	Enrollment	Register a person with one or more enrollment samples and create a versioned identity template.
FR-02	Face detection	Detect zero, one, or many faces in an image/frame and return bounding boxes and detector confidence.
FR-03	Alignment	Align each face using stable landmarks before embedding.
FR-04	Quality gate	Compute image-quality signals and reject or down-weight unusable crops.
FR-05	Embedding	Generate a normalized face embedding using the selected recognition model.
FR-06	Search	Return top-K candidate identities from a vector index.
FR-07	Unknown rejection	Return UNKNOWN when similarity/evidence is below the calibrated operating point.
FR-08	Decision fusion	Combine similarity, quality and temporal evidence into a final decision.
FR-09	Tracking	Maintain face track IDs across frames and avoid identity flicker.
FR-10	Liveness	Optionally require a liveness result before a security-sensitive acceptance.
FR-11	Audit	Persist decision metadata sufficient to reconstruct why a match was accepted/rejected.
FR-12	Model traceability	Persist model version and threshold profile used for each recognition event.
FR-13	Camera management	Register cameras, configure inference profile, and start/stop processing.
FR-14	Health	Expose service, model and camera health/telemetry endpoints.
FR-15	Deletion	Delete or revoke a person/template and support retention-driven deletion of biometric records.

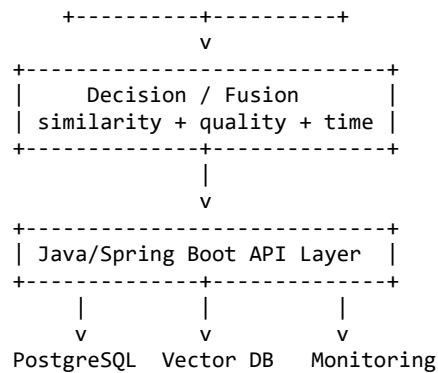
6. Non-Functional Requirements

ID	Area	Requirement
NFR-01	Accuracy	Optimize for low false-accept probability at a defined operating point; do not use raw accuracy as the sole release metric.
NFR-02	Latency	Measure p50, p95 and p99 inference and end-to-end decision latency.
NFR-03	Throughput	Measure sustained FPS per stream and maximum concurrent streams on target hardware.
NFR-04	Reliability	No single malformed frame should crash the processing loop or corrupt identity state.
NFR-05	Security	Authenticate APIs, authorize biometric operations, encrypt transport, and restrict

ID	Area	Requirement
		access by role.
NFR-06	Privacy	Minimize retained raw imagery; make retention and deletion configurable and auditable.
NFR-07	Observability	Metrics and structured logs must expose detector, quality, embedding, search and decision stages.
NFR-08	Reproducibility	Freeze dataset split, package versions, model hashes and configuration for each benchmark run.
NFR-09	Portability	Inference layer should support CPU and optional GPU execution and an ONNX deployment path.
NFR-10	Scalability	Gallery size and camera count must be explicit load-test dimensions.
NFR-11	Maintainability	Modules must have independent interfaces and unit/integration tests.
NFR-12	Safety of decisions	System must expose confidence, quality and UNKNOWN rather than forcing an identity for every detected face.

7. High-Level Architecture





Architecture principle: ML inference is exposed through a stable service contract so models can be benchmarked and replaced without rewriting the Java application layer.

8. Component Responsibilities

Module	Responsibility
Capture/ingestion	Frame acquisition, buffering, timestamps, frame sampling, back-pressure.
Face detector	Face boxes, detector score, optional landmarks.
Tracker	Track ID continuity across frames; associates detections over time.
Quality engine	Blur, exposure, face size, pose, occlusion and quality score.
Alignment	Canonical crop using landmarks; standardized input to recognizer.
Recognition engine	Embedding generation; model loading; batch inference; CPU/GPU selection.
Vector search	ANN index, top-K retrieval, gallery refresh and model-version isolation.
Decision engine	Thresholding, UNKNOWN rejection, temporal voting, candidate re-ranking, policy checks.
Liveness	Presentation-attack detection where required by the deployment policy.
Enrollment service	Capture/validate samples, quality filter, create templates and versions.
Java API	Identity/camera/admin APIs, auth, authorization, orchestration, audit and persistence.
Observability	Latency, FPS, error rate, quality reject rate, match/reject rate, drift indicators.
Configuration	Model registry, threshold profiles, camera profiles and retention settings.

9. Core Data Flow

1. Capture a frame and assign a monotonic frame ID plus timestamp.
2. Detect faces. If none are present, continue to the next frame.
3. Create/refresh tracks for detected faces. Avoid running full recognition on every frame if the tracker can safely reuse recent embeddings.
4. For each candidate crop, calculate quality signals. Reject very small, blurred, severely occluded or extreme-pose faces according to the configured policy.
5. Align the face using landmarks and generate a normalized embedding.
6. Query the vector index for top-K candidate templates belonging to the same model/version.
7. Combine similarity, quality and temporal evidence. Require a calibrated acceptance threshold; otherwise output UNKNOWN.
8. Optionally require liveness for security-sensitive decisions.
9. Persist a recognition event containing model version, threshold profile, candidate, scores, quality and decision reason.
10. Expose the result through the Java API/UI and emit operational telemetry.

10. Enrollment Design

Single-photo enrollment is explicitly rejected as the default strategy because it makes the gallery overly dependent on one pose, one lighting condition and one appearance state. Enrollment should create an identity template from multiple high-quality samples.

Enrollment stage	Rule
Capture	Collect a configurable set of samples spanning frontal, slight left/right pose and realistic expression/appearance variation.
Quality filtering	Discard samples below minimum size/quality or with severe blur/occlusion.
Embedding	Generate one embedding per accepted sample using the active recognition model.
Template	Keep multiple embeddings plus an optional prototype/centroid representation.
Metadata	Store sample quality, pose, model version and creation time.
Versioning	Never silently mix embeddings produced by incompatible recognition models.
Re-enrollment	Allow new samples to augment or replace the template under an explicit versioned operation.

11. Model Selection Strategy

The project should benchmark model families by role. Popularity, repository stars, or a vendor name are not sufficient selection criteria. ArcFace provides a strong face-recognition baseline based on additive angular margin learning. MagFace explicitly couples embedding learning with an interpretable quality signal, and AdaFace is designed to adapt learning to face-image quality. InsightFace provides a mature ecosystem spanning face detection, alignment and recognition implementations, but model/data licensing must be checked for the exact intended use. [R1-R3,R7]

Candidate	Role	Why test it	Decision
ArcFace / IResNet	Recognition baseline	Strong, widely reproduced embedding approach; clear reference baseline. [R1]	Baseline
AdaFace	Recognition	Quality-adaptive recognition; directly relevant to low-quality and appearance-variation experiments. [R2]	Primary research candidate
MagFace	Recognition + quality	Embedding magnitude is correlated with face quality; useful for quality-aware experiments. [R3]	Primary research candidate
InsightFace model zoo	Integrated face stack	Practical packaging and mature detection/recognition ecosystem. [R7]	Deployment candidate; verify licensing
SCRFD	Detection	Efficiency-oriented face detector candidate for real-time deployment.	Detection benchmark A
RetinaFace	Detection	Strong detection/landmark baseline for difficult faces.	Detection benchmark B
ByteTrack	Tracking	Fast multi-object tracking baseline for video identity persistence.	Tracking baseline
Liveness model	Anti-spoofing	Adds a security gate against presentation attacks.	Phase-2 security module

12. Model-Selection Experiment

12.1 Controlled Experimental Design

1. Create a subject-disjoint dataset split: no person appears in both development/training-tuning data and final test identities.
2. Freeze the test set before final tuning. Thresholds are learned only on a development set.
3. Keep detection, alignment and preprocessing constant while comparing recognition models; repeat the process with a fixed recognition model while comparing detectors.
4. Run both 1:1 verification and 1:N identification tests.
5. Run clean, controlled perturbation suites so specific failure modes can be attributed to lighting, blur, pose, beard, hairstyle, aging, camera domain and resolution.
6. Run each benchmark on the same hardware and software build. Record CPU/GPU model, runtime provider, batch size, image size and precision mode.
7. Run at least three repeated performance measurements after warm-up and report median plus p95/p99 where relevant.
8. Select the model using a weighted score dominated by false-accept behavior and robustness, with latency and memory as deployment constraints rather than arbitrary priorities.

12.2 Required Comparison Matrix

Dimension	Measure	Purpose
Verification accuracy	TAR, FNMR at operating FAR	Measures true-match acceptance while controlling false matches.
Identification accuracy	Rank-1, Rank-5, FNIR	Measures one-to-many identification quality and rejection.
False acceptance	FMR/FAR	Primary security metric; wrong identity accepted.
Unknown rejection	False positive identification rate / open-set DIR	Measures behavior when the person is not enrolled.
Robustness	Condition-wise TAR/FAR delta vs clean	Quantifies degradation from beard, aging, pose, light, blur, etc.
Quality behavior	Performance by quality bucket	Shows whether low-quality faces are causing failures.
Speed	ms/frame, FPS, p95 latency	Determines real-time feasibility.
Memory	Peak RAM/VRAM	Determines edge deployment feasibility.
Scalability	Search latency vs gallery size	Determines when ANN/vector indexing is required.
Stability	Identity switch rate per track	Measures temporal consistency in video.

13. Recognition Decision Policy

The decision layer must never equate “nearest candidate” with “recognized person.” Every query gets a candidate score, but acceptance requires passing a calibrated threshold and policy gates.

```
candidate = vector_search(query_embedding, top_k=K)
score = similarity(query_embedding, candidate.embedding)
quality_factor = quality_policy(face_quality, pose, size, blur)
temporal_factor = track_consensus(track_history)
liveness_ok = liveness_policy(result)
```

```
if score >= threshold and quality_factor >= q_min \
    and temporal_factor >= t_min and liveness_ok:
    decision = IDENTIFIED
else:
```

decision = UNKNOWN / REJECTED

The exact numerical thresholds must be learned from development data and validated on the frozen test set. They must not be hard-coded from a blog post or copied from another dataset.

14. Video and Temporal Logic

Recognition should be event-oriented rather than frame-oriented. The tracker can maintain a track state containing recent embeddings, candidate identities, scores and quality. A stable decision can be produced after multiple sufficiently good observations rather than trusting a single frame.

Track field	Purpose
track_id	Stable identifier for a detected face trajectory.
embedding_history	Recent high-quality embeddings for the track.
candidate_history	Top candidates and scores over recent frames.
quality_history	Quality of the evidence used for decisioning.
decision_state	UNKNOWN / TENTATIVE / IDENTIFIED / REJECTED.
last_seen	Expiry and lifecycle management.
model_version	Prevents mixing incompatible embedding spaces.

15. Database Schema

Relational storage should hold identities, operational configuration and audit data. Embeddings may be stored in PostgreSQL with pgvector or in a dedicated vector engine; the initial prototype can use PostgreSQL + FAISS if that is simpler. The relational schema below is the logical contract independent of the physical vector technology.

Table	Logical columns
persons	person_id PK; external_ref; display_name; status; created_at; updated_at; deleted_at
face_templates	template_id PK; person_id FK; model_id; model_version; template_type; quality_summary; created_at; active_from; retired_at
face_embeddings	embedding_id PK; template_id FK; vector; source_sample_id; quality_score; pose_score; created_at
enrollment_samples	sample_id PK; person_id FK; storage_ref; capture_time; source_type; quality_score; status; retention_deadline
cameras	camera_id PK; name; location_label; stream_uri_ref; profile_id FK; status; created_at
model_registry	model_id PK; version; role; artifact_ref; hash; runtime; input_shape; dimension; license_note; active
threshold_profiles	profile_id PK; model_id FK; verification_threshold; identification_threshold; quality_min; temporal_min; created_at
recognition_events	event_id PK; camera_id FK; track_id; person_id nullable; decision; similarity; quality_score; liveness_score; model_version; threshold_profile_id; timestamp; reason_code
audit_logs	audit_id PK; actor_id; action; object_type; object_id; timestamp; result; metadata
system_profiles	profile_id PK; detector_model; recognizer_model; tracker_model; sample_rate; retention_policy; config_json

16. API Specification

16.1 Java/Spring Boot API

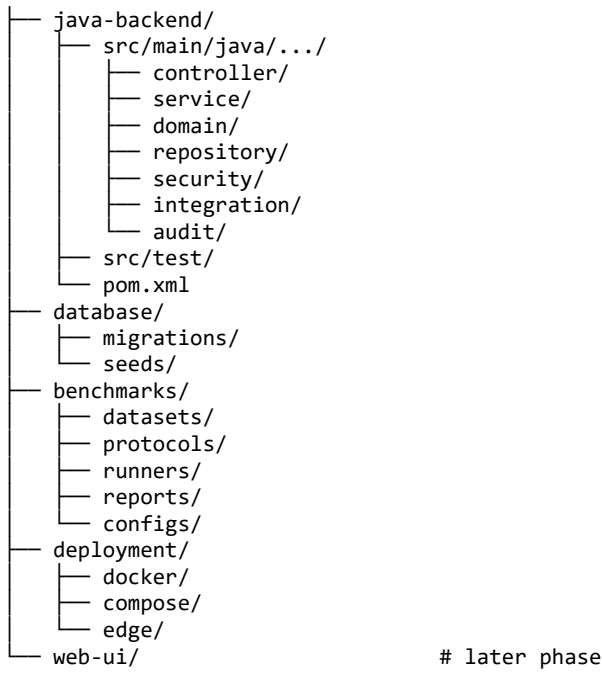
Method	Endpoint	Purpose
POST	/api/v1/persons	Create a person record.
GET	/api/v1/persons/{id}	Get person metadata/status.
DELETE	/api/v1/persons/{id}	Revoke/delete identity and associated biometric records according to retention policy.
POST	/api/v1/persons/{id}/enrollment	Submit enrollment samples and create a template version.
GET	/api/v1/persons/{id}/templates	List active/retired template versions.
POST	/api/v1/recognition/image	Run one-shot image recognition.
POST	/api/v1/recognition/verify	Verify claimed identity using 1:1 matching.
POST	/api/v1/cameras	Register a camera.
GET	/api/v1/cameras	List camera status/configuration.
POST	/api/v1/cameras/{id}/start	Start camera processing.
POST	/api/v1/cameras/{id}/stop	Stop camera processing.
GET	/api/v1/events	Query recognition/audit events with filters.
GET	/api/v1/health	Application health.
GET	/api/v1/metrics	Prometheus-compatible service metrics or aggregated telemetry.

16.2 Python Inference Service Contract

Endpoint / call	Input	Output
POST /infer/detect	image/frame	face boxes, landmarks, detector scores
POST /infer/embed	aligned face(s), model_version	embedding(s), quality metadata
POST /infer/recognize	embedding + search policy	top-K candidates, similarity scores
POST /infer/liveness	face crop/frame sequence	liveness score / status
GET /infer/models	none	loaded models, versions, hashes, runtime provider
GET /infer/health	none	service/model health

17. Folder / Repository Structure

```
face-recognition-system/  
├── README.md  
├── docs/  
│   ├── system-design.md  
│   ├── model-selection.md  
│   ├── benchmark-protocol.md  
│   └── threat-model.md  
├── python-vision/  
│   ├── app/  
│   │   ├── api/  
│   │   ├── detection/  
│   │   ├── tracking/  
│   │   ├── quality/  
│   │   ├── alignment/  
│   │   ├── recognition/  
│   │   ├── liveness/  
│   │   ├── search/  
│   │   └── decision/  
│   ├── models/  
│   ├── configs/  
│   ├── tests/  
│   └── requirements.txt
```



18. Exact Benchmark / Evaluation Methodology

This section is the release gate. A model is not considered “best” unless it wins on the required operating conditions while satisfying deployment constraints.

18.1 Dataset Construction

1. Collect or use a legally/ethically permissible dataset with multiple images per subject. Record source/capture metadata needed for condition analysis.
2. Create three identity-disjoint partitions: development/train, validation/tuning, and final test. No subject may cross partitions.
3. For the project-specific variation suite, define a fixed taxonomy: clean/frontal, low light, underexposure, overexposure/backlight, left/right pose, up/down pitch, blur, low resolution, occlusion, glasses, beard/no beard, hairstyle changes, weight/body-appearance change, time gap/aging, and camera-domain change.
4. Use real samples whenever available. Synthetic perturbations may supplement the suite but must be labeled as synthetic and should never be the only evidence for a failure mode.
5. Balance the test protocol enough that one easy condition does not dominate the aggregate score.

18.2 Verification Protocol (1:1)

Generate genuine pairs from the same subject and impostor pairs from different subjects. Evaluate similarity distributions and produce ROC/DET curves. For selected target FAR values, report TAR (or 1-FNMR).

Metric	Definition / reporting rule
TAR @ FAR	Primary release metric. Report at at least 1e-2, 1e-3, 1e-4, and a lower FAR when sample count permits.
FMR / FAR	Fraction of impostor comparisons accepted.
FNMR / FRR	Fraction of genuine comparisons rejected.
EER	Intersection point of false-match and false-non-match rates; secondary diagnostic only.
Threshold	Report the exact threshold used for each operating point.
Confidence intervals	Use bootstrap confidence intervals when dataset size allows.

18.3 Identification Protocol (1:N)

Create an enrolled gallery and probe queries. Include both enrolled identities and truly unknown identities for open-set evaluation. Report rank-1/rank-5, false-negative identification rate, and false-positive identification/open-set behavior. NIST's face-recognition evaluations explicitly distinguish one-to-one verification from one-to-many identification and examine computational cost and image-quality effects. [R4-R6]

Metric	Purpose
Rank-1 accuracy	Correct identity is top result.
Rank-5 accuracy	Correct identity appears in top 5.
FNIR	True identities present in gallery but not correctly identified.
Open-set false positive rate	Unknown query incorrectly assigned to an enrolled identity.
Search latency	p50/p95/p99 top-K search time by gallery size.
Gallery scaling	Repeat at 100, 1k, 10k, 100k identities or the largest feasible sizes.

18.4 Condition Robustness Matrix

Condition	Verification TAR	False match	Identification/open-set	Quality	Latency
Clean/frontal	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Low light	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Backlight/overexposure	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Left/right yaw	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Pitch variation	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Motion blur	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Low resolution	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Glasses	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Mask/partial occlusion	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Beard/no beard	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Hairstyle change	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Weight/appearance change	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Time gap / aging	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Different camera	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency
Unknown person	TAR@target FAR	FMR/FAR	Rank-1 or open-set FPR	Mean quality	p95 latency

Required analysis: report the absolute metric and the degradation from the clean baseline for every condition. The project's main research result should be expressed as "performance retained under variation," not only "best clean-dataset score." NIST notes that false negatives are strongly dependent on image quality, which supports treating quality as an explicit evaluation axis. [R5]

18.5 Video / Temporal Evaluation

Metric	Method
Identity switch rate	Count identity changes within a ground-truth track after stabilization window.
Track-level recognition rate	A track counts as correct when the final stabilized decision is correct.
Decision latency	Time from first acceptable observation to stable decision.
Frame-level accuracy	Diagnostic only; should not replace track-level metrics.
Unknown stability	UNKNOWN remains UNKNOWN for truly unknown tracks without oscillating through enrolled identities.

18.6 Performance Benchmark

1. Warm up the inference runtime and model for a fixed number of iterations before timing.
2. Measure detector-only, embedding-only, search-only and full-pipeline timings separately.
3. Record p50/p95/p99 latency, sustained FPS, peak RAM/VRAM and CPU/GPU utilization.
4. Test realistic stream settings rather than only single-image batch throughput.
5. Repeat across CPU and target GPU/edge hardware.
6. Run a scale test by increasing gallery size and camera count until a defined resource limit or SLA is reached.

18.7 Release Scorecard

Category	Weight / gate	Release rule
False acceptance / operating point	Hard gate	Must meet project FAR target on frozen test set.
Robustness	40% of composite score	No critical condition may collapse beyond defined tolerance vs clean baseline.
Verification / identification	25%	Must meet minimum TAR/rank-1 targets.
Latency / FPS	20%	Must meet target on selected hardware.
Memory / scalability	10%	Must remain within deployment budget.
Operational stability	5%	No critical crashes, uncontrolled identity switching or corrupt state.

The percentages above are a starting policy, not a claim that the system is universally optimized. For security-sensitive deployment, false acceptance remains a hard gate even when a weighted score looks good.

19. Acceptance Criteria for the First Production-Ready Prototype

- One-shot 1:1 verification and 1:N identification work through stable APIs.
- UNKNOWN is supported and calibrated; the system never forces the nearest gallery identity when evidence is insufficient.
- Multiple enrollment samples materially outperform single-image enrollment on the project's held-out appearance-variation suite.
- At least two recognition candidates (for example ArcFace/AdaFace/MagFace) are benchmarked under identical conditions before final selection.
- At least two detectors (for example SCRFD/RetinaFace) are benchmarked for recall, latency and robustness.
- Video tracking and temporal consensus reduce identity flicker relative to frame-by-frame recognition.
- Model versions, thresholds and event metadata are persisted for reproducibility.
- Privacy, deletion and audit paths are implemented before any broader deployment.

20. Security, Privacy and Responsible-Use Controls

- Use the system only in authorized contexts with an explicit data-governance basis appropriate to the deployment.
- Minimize raw-image retention; retain embeddings and event metadata only as required by the defined purpose and retention policy.
- Encrypt biometric data at rest and in transit; restrict administrative operations by role.
- Provide deletion/revocation of identity templates and associated biometric records.
- Log enrollment, template changes, model changes, threshold changes and recognition-policy changes.
- Keep an audit trail that identifies model/version and decision policy used for security-sensitive outcomes.
- Do not infer sensitive demographic attributes unless there is a separate, justified and reviewed requirement; they are not part of this core system.

21. Risks and Mitigations

Risk	Mitigation
Appearance changes break recognition	Use multiple enrollment samples, quality-aware models, temporal evidence and condition-specific testing.
Low-quality faces create false matches	Quality gate + calibrated threshold + UNKNOWN rejection.
Model overfits one benchmark	Subject-disjoint project-specific test suite and cross-camera evaluation.
Model license mismatch	Record license in model registry and verify intended commercial/research use before deployment.
Embedding version incompatibility	Version every embedding and template; support migration/re-enrollment.
Frame-by-frame identity flicker	Track IDs and temporal decision fusion.
Photo/video spoofing	Add liveness/anti-spoofing where security risk requires it.
Gallery search becomes slow	ANN/vector search and explicit gallery-scale benchmarks.
Privacy exposure	Minimize images, encrypt data, RBAC, auditing and deletion workflows.
False confidence from one high similarity score	Require threshold + quality + temporal policy rather than score alone.

22. Implementation Phases

Phase	Focus	Exit result
Phase 0	Environment + dataset + benchmark harness	Freeze repository, dataset splits, metrics and baseline scripts.
Phase 1	Offline recognition core	Detection -> alignment -> embedding -> similarity -> UNKNOWN.
Phase 2	Robustness layer	Multi-sample enrollment, quality gate, threshold calibration, condition suite.
Phase 3	Real-time video	Tracking, temporal consensus, streaming pipeline.
Phase 4	Backend platform	Spring Boot, PostgreSQL, APIs, auth, audit, configuration.
Phase 5	Security	Liveness, retention/deletion, threat model, hardening.
Phase 6	Optimization	ONNX Runtime/TensorRT where appropriate, batching, caching, ANN search.
Phase 7	Multi-camera + edge	Concurrent stream load tests, edge deployment, monitoring and operational tooling.

23. Open Decisions to Resolve Before Coding the Production Stack

- Target deployment hardware for the first benchmark: CPU-only laptop, NVIDIA GPU desktop, or a specific edge device.
- Initial gallery size and camera count target.
- Primary operating mode: 1:1 verification, 1:N identification, or both.
- Application policy for unknowns and the acceptable false-accept operating point.
- Whether the first implementation stores vectors in PostgreSQL/pgvector or uses a separate ANN/vector service.
- Exact dataset and data-collection policy, including consent/authorization and retention.

24. Recommended Initial Technology Baseline

Layer	Baseline	Alternative / later
Language	Python for ML; Java for backend	All-Python prototype or separate inference microservice in another runtime
ML runtime	PyTorch during research; ONNX Runtime for deployment benchmark	TensorRT on NVIDIA edge/GPU
Detection	SCRFD + RetinaFace benchmark	YOLO-family face detector if it wins deployment tests
Recognition	ArcFace baseline; AdaFace and MagFace experiments	Other current models added through the same inference interface
Tracking	ByteTrack	DeepSORT or another tracker if benchmarked superior
Database	PostgreSQL	PostgreSQL + pgvector / separate vector DB at scale
Backend	Spring Boot	FastAPI for small prototypes
Deployment	Docker Compose	Kubernetes / edge-specific deployment later
Monitoring	Prometheus-compatible metrics	Grafana + centralized logs

25. Definition of “Done” for the Architecture Stage

The architecture stage is complete when the repository contains: (1) this specification, (2) frozen benchmark protocol and data split definitions, (3) model abstraction interfaces, (4) database migration plan, (5) API contracts, (6) a reproducible offline baseline benchmark, and (7) a pass/fail checklist for the first implementation milestone. No production optimization should begin before the baseline benchmark can measure whether an optimization actually improves the system.

26. References

[R1] Deng et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” CVPR 2019. Open access: https://openaccess.thecvf.com/content_CVPR_2019/html/Deng_ArcFace_Additive_Angular_Margin_Loss_for_Deep_Face_Recognition_CVPR_2019_paper.html

[R2] Kim et al., “AdaFace: Quality Adaptive Margin for Face Recognition,” CVPR 2023. Use the paper/repository version selected for the experiment and record the exact checkpoint/hash in the model registry.

[R3] Meng et al., “MagFace: A Universal Representation for Face Recognition and Quality Assessment,” CVPR 2021. https://openaccess.thecvf.com/content/CVPR2021/html/Meng_MagFace_A_Universal_Representation_for_Face_Recognition_and_Quality_Assessment_CVPR_2021_paper.html

[R4] NIST Face Recognition Vendor Test / Face Recognition Technology Evaluation overview:

<https://www.nist.gov/programs-projects/face-recognition-vendor-test>

[R5] NIST FRTE Demographic Effects summary, including quality-related false-negative observations:

https://pages.nist.gov/frvt/html/frvt_demographics.html

[R6] NIST Face Identification evaluation report (NISTIR 8009), covering accuracy and computational resource usage:

<https://www.nist.gov/publications/face-recognition-vendor-test-frvt-performance-face-identification-algorithms-nist-ir>

[R7] InsightFace project and model licensing notes: <https://github.com/deepinsight/insightface>